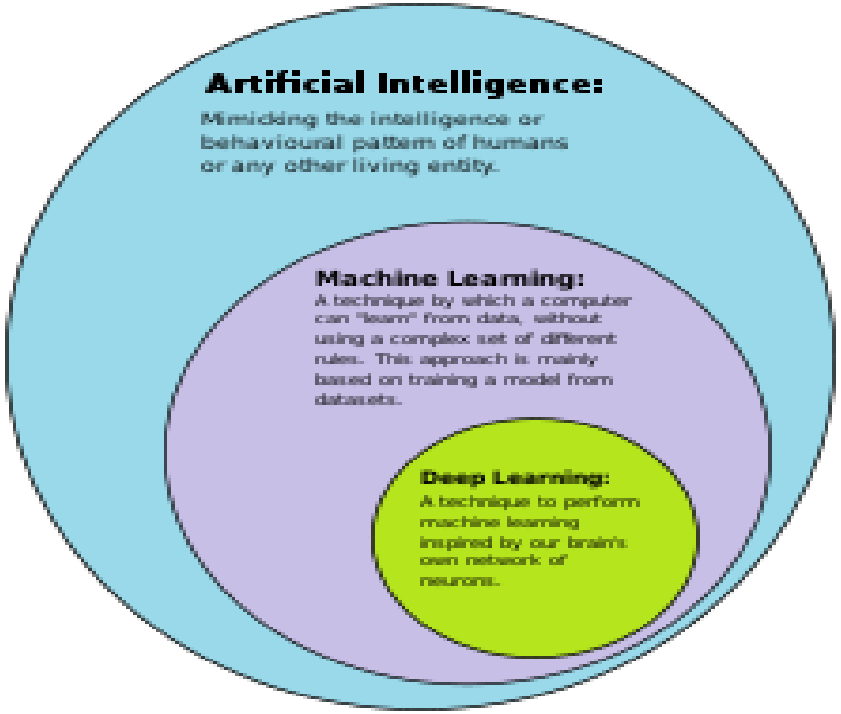
Sección 5: Machine Learning

Contexto IA


Artificial Intelligence:
Mimicking the intelligence or behavioural pattern of humans or any other living entity.
Machine Learning:
A technique by which a computer can "learn" from data, without using a complex set of different rules. This approach is mainly based on training a model from datasets.
Deep Learning:
A technique to perform machine learning inspired by our brain's own network of neurons.

¿Qué es el Machine Learning?

- El aprendizaje Automático es un subdominio de la IA que proporciona a los sistemas, la capacidad de aprender y mejorar automáticamente a partir de la experiencia, sin ser programados explícitamente (Arthur Samuel 1959). Se basa en la hipótesis subyacente de crear el modelo y tratar de mejorarlo ajustando más datos en el modelo a lo largo del tiempo.
- Experiencia (datos) -> Modelo -> Predicciones

¿Cuándo utilizar ML?

- En soluciones que funcionan mediante la aplicación de un conjunto extenso de reglas o heurísticas
- En problemas complejos, en los que el analista no es capaz de determinar una solución a partir de la información existente
- En entornos que fluctúan o varían con frecuencia
- Apoyo de la fase de análisis en enfoques tradicionales en los que se dispone de conjuntos de datos muchos más grandes y difíciles de interpretar.

Aprendizaje supervisado

- Utiliza datos etiquetados (pares de entrada-salida) para entrenar modelos
- La regresión predice valores continuos (por ejemplo, precios de viviendas).
- La clasificación o regresión logística, predice valores discretos (por ejemplo, si un mensaje es spam o no).

Aprendizaje no supervisado

- Infiere una función para describir la estructura de datos sin etiquetar, a diferencia del aprendizaje supervisado, que realiza predicciones a partir de datos etiquetados.
- Diferencia clave: El aprendizaje no supervisado se centra en describir la estructura de los datos, no en realizar predicciones.
- Aplicación: Resulta útil para analizar grandes conjuntos de datos sin etiquetar, con el fin de encontrar patrones y obtener información valiosa.

Aprendizaje online y batch

- Aprendizaje por lotes: Se entrena con todos los datos disponibles a la vez, lo que requiere reentrenamiento con nuevos datos. Es más sencillo, pero menos adaptable.
- Aprendizaje en línea: Se entrena de forma incremental con puntos de datos individuales o minilotes, lo quepermite la adaptación continua y el manejo de grandes conjuntos de datos. Sin embargo, puede ser inestable con datos de baja calidad y requiere prefiltrado.

Aprendizaje basado en instancias y basado en modelos

- El aprendizaje basado en instancias utiliza medidas de similitud para comparar nuevos datos con ejemplos de entrenamiento almacenados.
- El aprendizaje basado en modelos construye una función de hipótesis (un modelo matemático) para realizar predicciones.

Sección 6: Regresión Lineal

Regresión Lineal: Introducción

- Es un algoritmo de aprendizaje supervisado, basado en modelos, que predice valores continuos.
- El algoritmo construye una función de hipótesis lineal para modelar datos con una tendencia lineal.
- Realiza predicciones computando una suma ponderada de las características de entrada y sumándole una constante conocida como Bias o sesgo.
- Este algoritmo intenta predecir valores continuos, clasificándose dentro de las técnicas de regresión en el aprendizaje supervisado.
- Su objetivo es construir una función hipótesis que se ajuste de la mejor manera posible a la tendencia lineal de los datos de entrenamiento.

- Un ejemplo de su uso es predecir el coste de un incidente de seguridad basándose en el número de sistemas afectados. En este caso, el algoritmo utilizaría datos de incidentes pasados (datos etiquetados) que incluyen el número de equipos afectados (característica de entrada) y el coste que tuvo ese incidente (característica de salida o valor a predecir). La salida del algoritmo sería una función matemática lineal que representa la tendencia de estos datos, permitiendo predecir el coste de futuros incidentes.

- En una regresión lineal, las características de entrada o variables de entrada se denotan siempre con una X. Si hay una única característica de entrada, se representa como X1. Si hay más de una característica de entrada, se utilizan X1, X2, X3 y así sucesivamente hasta XN, donde N es el número de características o variables de entrada.
- La variable de salida se corresponde con la letra Y. Normalmente es una única variable de salida, aunque algunos algoritmos permiten realizar predicciones múltiples, en cuyo caso se usarían Y1, Y2, etc.
- Un ejemplo de entrenamiento se corresponde con la tupla (X, Y), donde X puede ser un conjunto de características de entrada (X1, X2, ..., XN) e Y es el valor de salida

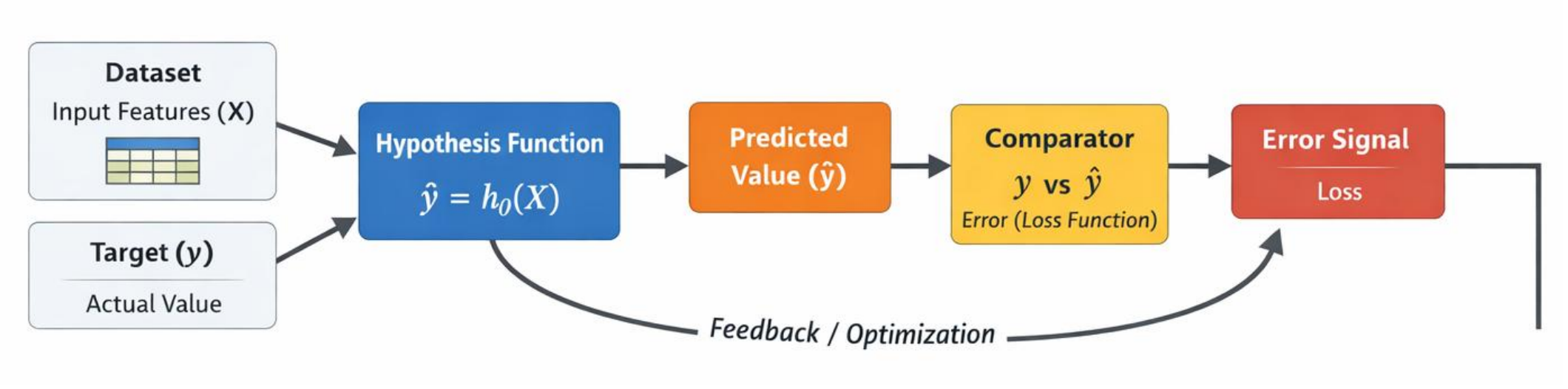
Regresión Lineal: Función hipótesis

- La función hipótesis es la función matemática asociada con la regresión lineal. Esta función se ajusta a la tendencia de los datos del conjunto de datos, dando como resultado un modelo. El modelo, que es la función hipótesis ajustada, se utiliza posteriormente para realizar predicciones.
- En el caso de la regresión lineal univariable, la función hipótesis es la función de la recta, que se representa como $H(x) = \theta_0 + \theta_1 x$. Aquí, θ_0 y θ_1 son los parámetros del modelo. θ_0 determina el punto de corte con el eje Y, y θ_1 indica la pendiente de la recta. El objetivo es encontrar los valores óptimos de θ_0 y θ_1 que ajusten la recta lo mejorposible a la tendencia de los datos.
- Cuando hay múltiples características de entrada (regresión lineal multivariable), la función hipótesis cambia a $H(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + ... + \theta n xn$. En este caso, hay más parámetros del modelo (θ_0, θ_1, θ_2, ..., θn), donde θ_0 es el término de sesgo (bias) y θ_1, θ_2, ..., θn son los pesos o ponderaciones que ponderan las características de entrada (x_1, x_2, ..., xn).
- En resumen, la función hipótesis es una suma ponderada de las entradas, donde los parámetros del modelo (θ) son los pesos que determinan la forma específica de la función matemática que se ajustará a los datos.

Regresión Lineal: Construcción del Modelo

- El principal desafío consiste en encontrar los valores óptimos de θ_0 y θ_1 que mejor se ajusten a los datos de entrenamiento.
- Esto se logra mediante una <u>función de costo</u> para medir los errores de predicción y una <u>función de optimización</u> para minimizarlos; este <u>proceso</u> se denomina <u>entrenamiento</u>.

Diagrama de flujo de aprendizaje supervisado



Proceso general de entrenamiento de un algoritmo de machine learning

1. **Inicialización de parámetros**: Se inicializan los parámetros θ_0 y θ_1 de manera aleatoria.

2. **Cálculo del error con la función de coste**: Se utiliza una función de coste o función de error para calcular el error de la función hipótesis ajustada con los valores aleatorios de los parámetros, en relación con los ejemplos del conjunto de datos. Este cálculo requiere que el conjunto de datos esté etiquetado, es decir, se necesita conocer la variable de salida de los datos.

3. **Minimización del error con la función de optimización**: Se emplea una función de optimización para minimizar este error. Esta función se encarga de modificar los parámetros θ_0 y θ_1 de manera que la función hipótesis se ajuste lo mejor posible a la tendencia de los datos.

4. **Iteración hasta la convergencia**: El proceso de modificar los parámetros y calcular el error continúa hasta que el error que proporciona la función de coste ya no pueda disminuirse más. En ese punto, la función hipótesis se habrá ajustado lo mejor posible a la tendencia de los datos, y se considera que el modelo ha sido construido

Estrategias de cálculo y actualización

Método	Cómo se calcula el error	Cuándo se actualizan los parámetros	Ventajas	Desventajas
Batch Gradient Descent	Se calcula el error con todas las instancias del dataset y se obtiene un promedio.	Una vez por época (epoch)(al terminar de evaluar todos los datos).	Preciso y estable.	Lento con conjuntos grandes.
Stochastic Gradient Descent (SGD)	Se calcula el error para una sola instancia.	Después de cada instancia.	Rápido y eficiente en datasets grandes.	Actualizaciones ruidosas y menos estables.
Mini-Batch Gradient Descent	Se calcula el error por grupos pequeños (por ejemplo, 32 o 100 instancias).	Una vez por cada mini-batch.	Equilibrado entre precisión y velocidad.	Requiere elegir bien el tamaño del lote.

Esquema resumido

Batch Gradient Descent:
[Datos completos] → Calcular error medio → Actualizar parámetros → Repetir

Stochastic Gradient Descent:
Instancia 1 → Actualizar
Instancia 2 → Actualizar
Instancia 3 → Actualizar
...
→ Repetir

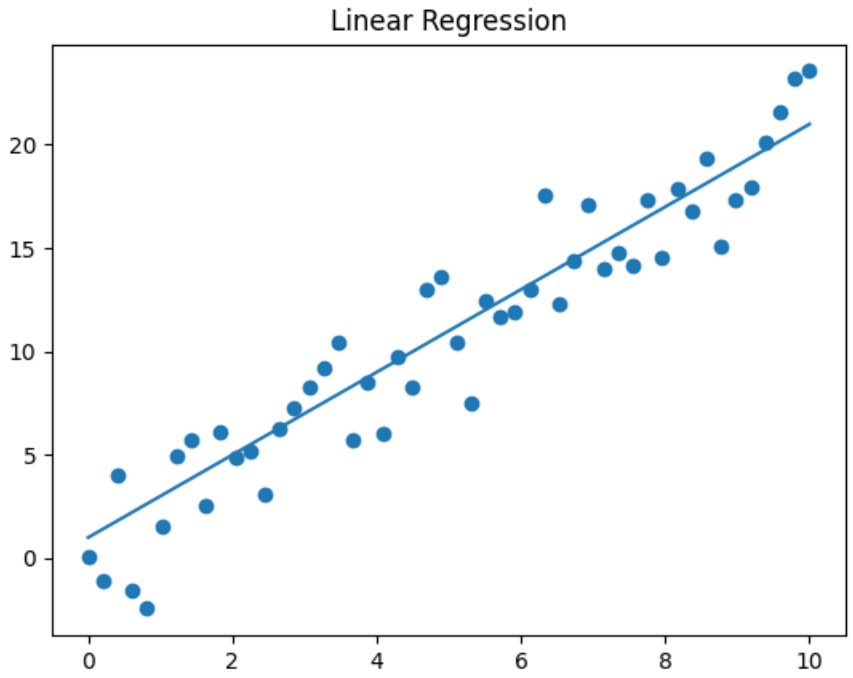
Mini-Batch Gradient Descent:
Lote 1 (ej. 32 muestras) → Calcular error medio → Actualizar
Lote 2 (32 muestras) → Calcular error medio → Actualizar
...
→ Repetir

La salida y calculada, se denota por ^y

Conclusión en el **proceso** de regresión lineal:

- Si el error se calcula sobre todo el conjunto de datos antes de ajustar los parámetros, se trata de Batch Gradient Descent.
- Si se calcula el error y se actualizan los parámetros por cada instancia, se trata de Stochastic Gradient Descent.
- En la práctica, la mayoría de los frameworks modernos (TensorFlow, PyTorch, Keras) usan mayormente Mini-Batch Gradient Descent, que combina lo mejor de ambos enfoques, que promedia el error de pequeños grupos de instancias y actualiza después de cada mini-lote.

Linear Regression



Regresión Lineal: Función de coste (J) Error Cuadrático Medio (ECM o MSE en inglés).

- El ECM ($J(\theta)$) cuantifica la precisión con la que una función de hipótesis predice los datos.
- Calcula la diferencia cuadrática media entre los valores predichos y los reales para todos los ejemplos de entrenamiento.
- Este error se minimiza mediante una función de optimización para encontrar los mejores parámetros (θ_0, θ_1) para la hipótesis.

Ecuaciones Regresión Lineal

1. Modelo Lineal

$h\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$

2. Forma Vectorial

$h\theta(x) = \theta^T x$

donde $x = (1, x_1, x_2, \ldots, x_n)$ y $\theta = (\theta_0, \theta_1, \ldots, \theta_n)$

3. Función de Costo (Error Cuadrático Medio)

$J(\theta) = (1 / 2m) * \Sigma (h\theta(x(i)) - y(i))^2$

4. Interpretación

El modelo predice un valor continuo como una combinación lineal de las características de entrada. Los parámetros θ se ajustan minimizando la función de costo J(θ).

Se calcula de la siguiente manera:

1. Inicialización de parámetros: El algoritmo de regresión lineal comienza inicializando los parámetros θ_0 y θ_1(para regresión lineal univariable) de manera aleatoria, lo que genera una función hipótesis.
2. Cálculo de la diferencia: Para cada ejemplo en el conjunto de datos de entrenamiento, se resta la etiqueta real (Yi) de la predicción de la función hipótesis (Hθ(Xi)).
3. Elevación al cuadrado: El resultado de la resta se eleva al cuadrado para asegurar que el error sea siempre un valor positivo y para penalizar más los errores grandes.
4. Suma de errores: Se suman todos los errores cuadrados de cada ejemplo en el conjunto de datos.
5. Promedio: La suma total de los errores cuadrados se divide por el número total de ejemplos (m), y generalmente también por 2 (2m) porque en la práctica funciona un poco mejor.

- La ecuación de la función de coste (J(θ)) es:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2$$

Regresión Lineal: Función de optimización (Descenso de gradiente)

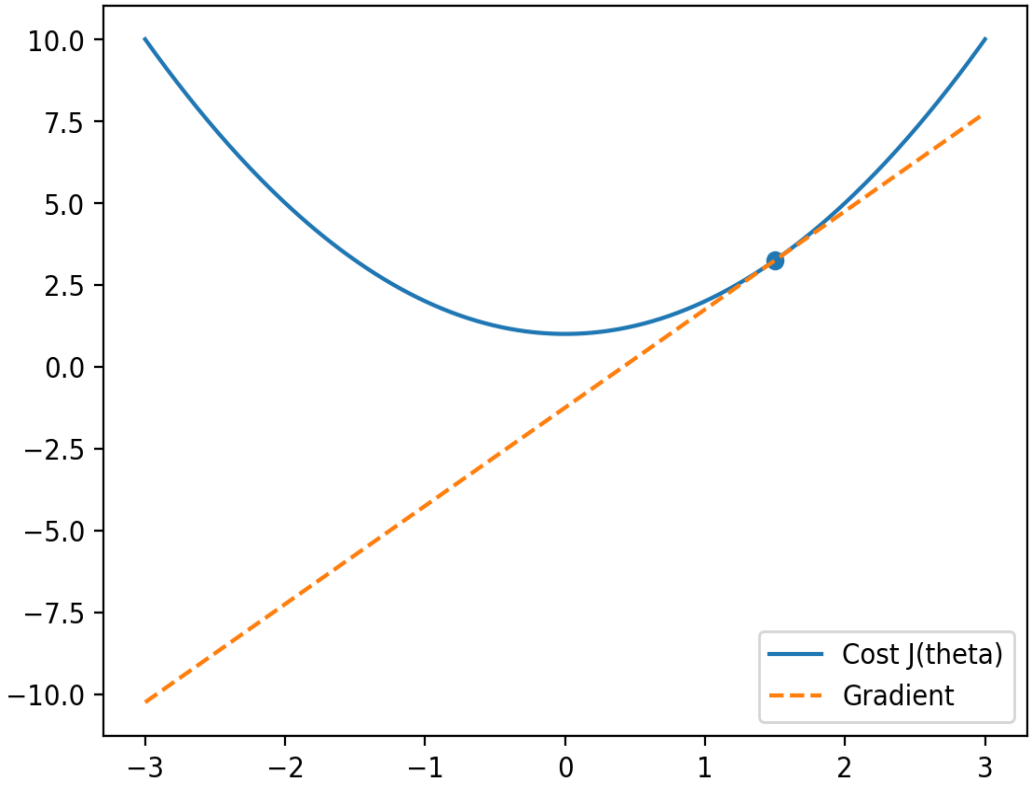
1. Inicialización aleatoria: Los parámetros se inicializan con un valor aleatorio.
2. Cálculo de la derivada: Se calcula la derivada parcial de la función de coste con respecto a los parámetros en el punto actual. La derivada indica la pendiente de la función en ese punto.
3. Actualización de los parámetros: Los parámetros se actualizan restando el producto de una constante llamada "learning rate" y la derivada de la función de coste. La fórmula general para la actualización de un parámetro (θ) es:

$$\theta = \theta - (\text{learning rate} \times dJ(\theta)/d\theta)$$

4. Iteración: Este proceso se repite hasta que el valor de los parámetros no varíe significativamente, lo que indica que se ha alcanzado el punto mínimo de la función de coste.

El "learning rate" es una constante que se le debe indicar de antemano al algoritmo de gradient descent. Generalmente, su valor suele ser 0.5 (un medio), aunque puede modificarse. Su rol es indicar la velocidad con la que el algoritmo se acerca al punto óptimo.

Gradient Descent



El descenso por gradiente es un algoritmo de optimización que ajusta los parámetros del modelo.
En cada iteración se actualizan los parámetros en la dirección que reduce el error.

Ejercicio Regresión Lineal

- Practicar cuaderno:

4_Regresión Lineal - Predicción del coste de un incidente de seguridad_CA.ipynb

Sección 7: Regresión Logística y Clasificación

Regresión Logística: Introducción

- La regresión logística predice valores discretos, a diferencia de la regresión lineal, que predice valores continuos.
- Utiliza un modelo lineal generalizado y aplica una función logística a la suma ponderada de las características de entrada y un sesgo.
- La principal aplicación que se analiza es la clasificación binaria, como la detección de spam, donde la salida es 0 o 1.

Regresión Logística

1. Modelo Lineal

$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$

2. Forma Vectorial

$z = \theta^T x$

donde $x = (1, x_1, x_2, \ldots, x_n)$ y $\theta = (\theta_0, \theta_1, \ldots, \theta_n)$

3. Modelo de Regresión Logística

$P(y=1|x) = \sigma(\theta^T x)$

Interpretación $P(y=1|x) = \sigma(\theta^T x)$

- $P(y=1|x)$ Es la **probabilidad** de que la salida sea 1 (clase positiva), dado un vector de entrada x. Ejemplo:

y=1: “el paciente tiene la enfermedad”

y=0: “no la tiene”

$\theta^T x$ Es un **modelo lineal**, igual que en regresión lineal:

$\Theta^T x = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + ... + \theta_n x_n$

Esto genera un valor que puede ser:

- negativo
- positivo
- cualquier número real
- **Problema**

Ese valor **no es una probabilidad** (puede ser mayor que 1 o menor que 0).

Solución: La Función Sigmoide

$\sigma(z) = 1 / (1 + e^{-z}) = H(x)$ <u>Función Hipótesis para la Regresión Logística</u>

Esta función transforma el valor z en una probabilidad entre 0 y 1.

Interpretación $P(y=1|x) = \sigma(\theta^T x)$

Intuición importante

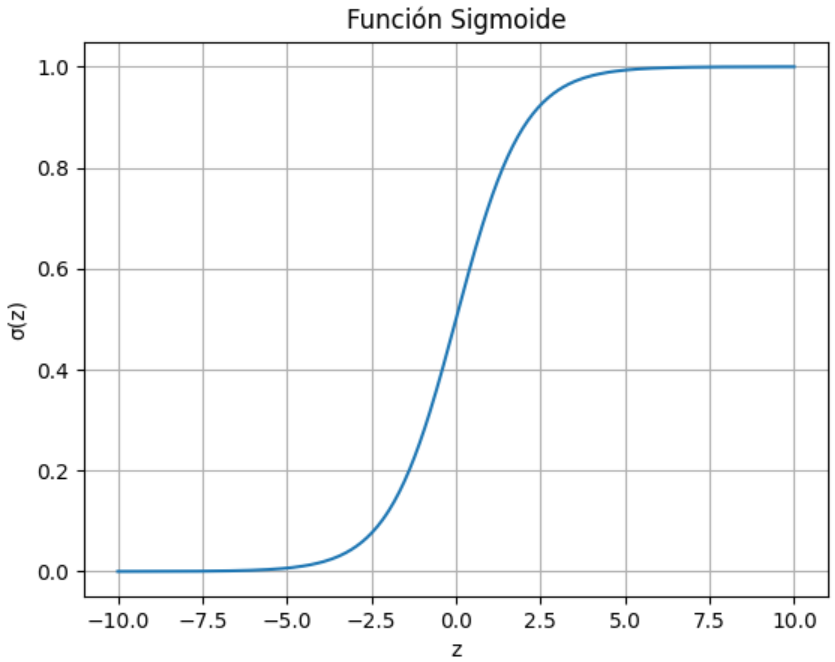
- Si $z \gg 0 \rightarrow \sigma(z) \approx 1 \rightarrow$ alta probabilidad de clase 1
- Si $z \ll 0 \rightarrow \sigma(z) \approx 0 \rightarrow$ baja probabilidad
- Si $z=0 \rightarrow \sigma(z)=0.5$

Es decir:

- La sigmoide convierte una **recta** en una **curva en forma de S**

En resumen: “La regresión logística toma una combinación lineal de las variables y la transforma en una probabilidad usando una función sigmoide.”

Regresión Logística: Función Sigmoide



La función sigmoide transforma valores en probabilidades entre 0 y 1.
Esto permite interpretar la salida del modelo como la probabilidad de pertenecer a una clase.
Donde.
$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$
$\sigma(z) = 1 / (1 + e^{-z})$ Función sigmoide

Interpretación:
- Cuando $z = 0 \rightarrow \sigma(z) = 0.5$
- Cuando $z > 0 \rightarrow \sigma(z)$ se acerca a 1
- Cuando $z < 0 \rightarrow \sigma(z)$ se acerca a 0

Ejemplo práctico: probabilidad de aprobar un examen

Supongamos que queremos estimar la probabilidad de aprobar un examen en función de las horas de estudio (x).

Modelo logístico: $p = 1 / (1 + e^{-z})$

Asumimos $\theta_0 = -2$ y $\theta_1 = 0.8$.

Caso 1: x = 5 horas → $z = -2 + 0.8(5) = 2$ → $p \approx 0.88$

Caso 2: x = 2 horas → $z = -2 + 0.8(2) = -0.4$ → $p \approx 0.40$

- **Referencias**

- Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications.
- Hastie, T., Tibshirani, R., & Friedman, J. (2017). The Elements of Statistical Learning (2nd ed.). Springer.
- Scikit-learn Documentation: Generalized Linear Models.
- Wikipedia contributors. Generalized linear model.

Regresión Logística: Representación de la función hipótesis

Limitaciones del uso de la función de hipótesis de la regresión lineal para problemas de clasificación.

- La función de hipótesis de la regresión lineal produce valores continuos, lo que dificulta la interpretación directa para la clasificación binaria (spam/no spam).
- Introducir un umbral (p. ej., 0.5) ayuda, pero los valores atípicos (outliers) pueden distorsionar la función lineal y provocar clasificaciones erróneas.
- La función también puede predecir valores fuera del rango deseado de 0 a 1, lo que requiere una nueva función de hipótesis para la regresión logística.

Regresión Logística: Función de coste

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\sigma(\theta^T x^{(i)}) + (1-y^{(i)})\log(1-\sigma(\theta^T x^{(i)}))\right]$$

¿Qué pasa con la ecuación de J cuando y = 0 o y = 1?

El objetivo es minimizar la función de coste J para encontrar lo parámetros óptimos.

En resumen, la función de coste para la regresión logística tiene dos partes, una para cuando la etiqueta real es 1 y otra para cuando la etiqueta real es 0. Ambas partes están diseñadas para que el coste sea alto cuando el algoritmo realiza una mala predicción y bajo cuando realiza una buena predicción.

La no linealidad de la sigmoide está en la función de activación, pero lo que determina la existencia o no de mínimos locales es la convexidad de la función de costo.

Como la función de costo es convexa, la regresión logística NO tiene múltiples <u>mínimos locales</u>. En una función convexa, todos los mínimos locales coinciden con el <u>mínimo global</u>.

Regresión Logística: Algoritmo de optimización

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\sigma(\theta^T x^{(i)}) + (1-y^{(i)})\log(1-\sigma(\theta^T x^{(i)}))\right]$$

- **Regla general de actualización:** $\theta_j \leftarrow \theta_j - \alpha\frac{\partial J(\theta)}{\partial\theta_j}$
- **Gradiente de la función de costo:**

$$\frac{\partial J(\theta)}{\partial\theta_j} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$$

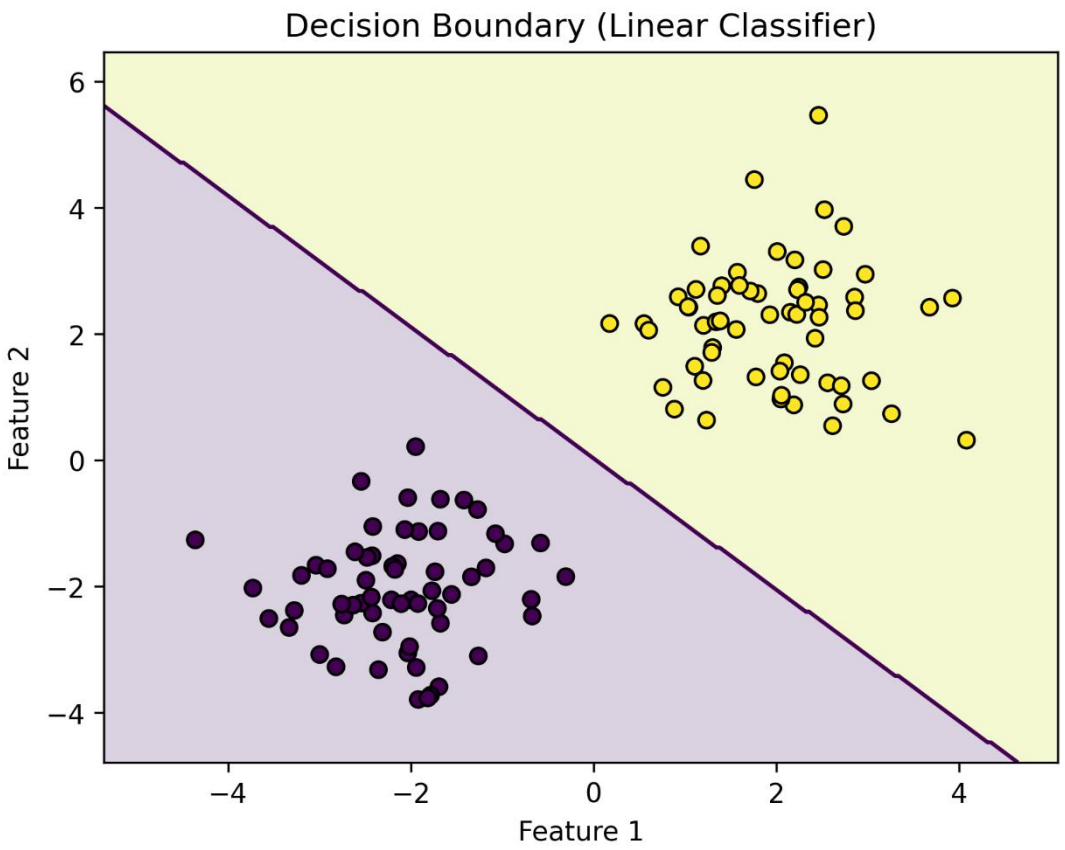
- **Regla de actualización completa:**

$$\theta_j \leftarrow \theta_j - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$$

- *Nota: α representa el learning rate o tasa de aprendizaje. Se denominará un hiperparámetro*

- La ecuación $\theta^T x = 0$ define una **frontera de decisión**
- Esta frontera es una:
  - **recta** (en 2D)
  - **plano** (en 3D)
  - **hiperplano** (en dimensiones mayores)
- Esa línea es la que separa las clases en la siguiente gráfica.
  - Un lado de la recta es para: $\theta^T x > 0$, $\sigma(z) > 0.5$, o sea Clase 1
  - El otro lado de la recta es para: $\theta^T x < 0$, $\sigma(z) < 0.5$, o sea Clase 0

- La sigmoide define la probabilidad, pero la recta define la decisión

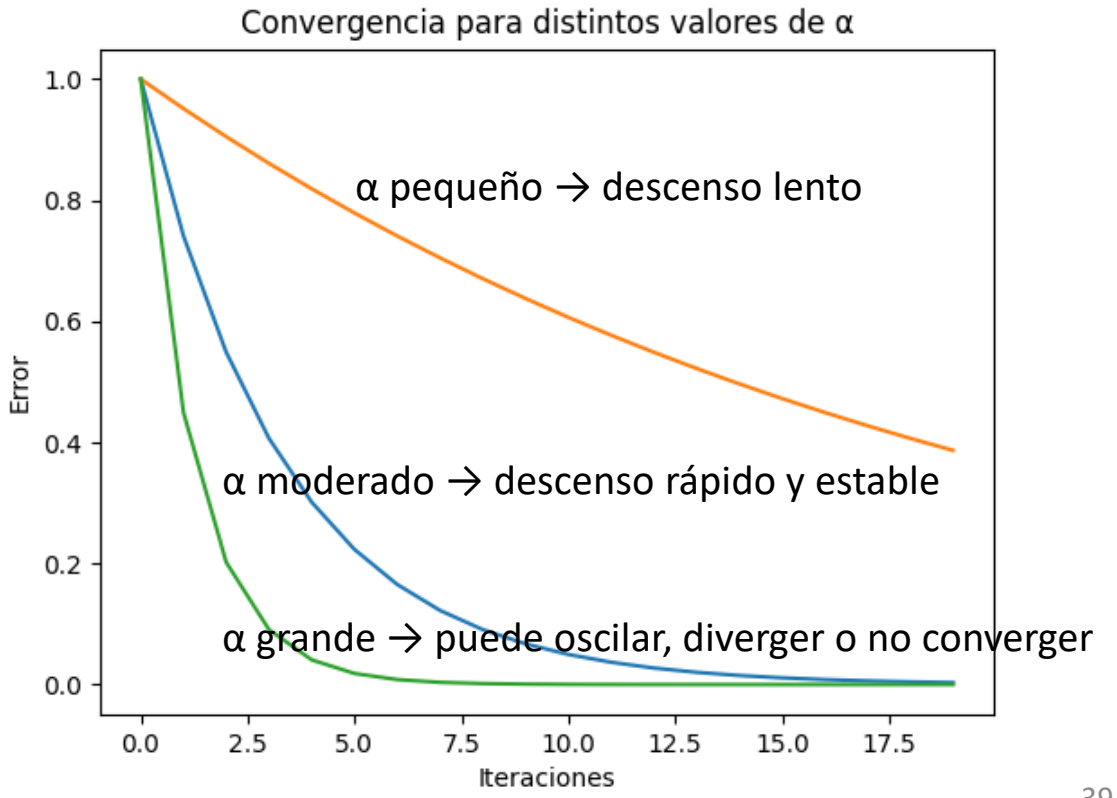

Decision Boundary (Linear Classifier)
Feature 2
Feature 1
6
4
2
0
−2
−4
−4
−2
0
2
4

Concepto de Hiperparámetro en ML

"Los parámetros se aprenden; los hiperparámetros se eligen."

θ → qué aprende el modelo
α → qué tan rápido aprende

Impacto de α en la convergencia



Ejercicio Regresión Logística

- Practicar cuaderno:

5_Regresión Logística - Detección de SPAM_CA.ipynb

Sección 8: Etapas de un proyecto de ML

Consideraciones

- Si bien algoritmos como la regresión lineal y logística son fundamentales, la mayor parte del tiempo en proyectos reales se dedica al procesamiento y la transformación de datos.
- La calidad y la cantidad de datos son tan importantes como el propio algoritmo para un aprendizaje automático eficaz.
- Si se tiene un conjunto de datos lo suficientemente grande y representativo del problema, la selección del algoritmo se convierte en un problema casi secundario, ya que muchos algoritmos se comportarán con una precisión y rendimiento similar al predecir nuevos ejemplos.

Recomendaciones claves para recopilar y preparar un conjunto de datos de entrenamiento

- **Disponer de una cantidad suficiente de datos**: Cuantos más datos se tengan, mejor se comportarán los algoritmos de machine learning. Con una mayor cantidad de datos, la selección del algoritmo se vuelve menos crítica, ya que la mayoría de los algoritmos alcanzarán un rendimiento y precisión similares, y este rendimiento será mayor cuanto más datos se utilicen.
- **Utilizar un conjunto de datos representativo del problema práctico**: Es esencial que las características de entrada o atributos extraídos sean representativas para diferenciar, por ejemplo, una transacción fraudulenta de una legítima. Extraer atributos que no tienen relación con el problema o que no marcan una diferencia significativa entre los tipos de ejemplos puede llevar a un peor rendimiento del algoritmo.
- **Los datos deben ser de calidad**: El conjunto de datos debe consistir en datos reales y no en datos generados manualmente o de forma artesanal. Utilizar datos no reales o de baja calidad impedirá que el algoritmo prediga correctamente para situaciones reales.
- **Seleccionar un conjunto de características adecuado, minimizando el número de características irrelevantes**: Cuantas más características de entrada se tengan, más recursos computacionales necesitará el algoritmo y más complejo será el modelo. Si se incluyen muchas características irrelevantes, el rendimiento del algoritmo puede deteriorarse y los recursos computacionales necesarios para el entrenamiento aumentarán considerablemente. Es importante seleccionar características que sean <u>significativas</u> para el problema.